PROJECT HANDBOOK

AI INTERVIEW COACH

Technical Architecture • Technology Stack • Project Structure

PROJECT PURPOSE

A web application designed to help students, graduates, and job seekers practise interviews, receive structured feedback, and identify areas for improvement.

Prepared for academic review

github.com/Mennatullah-essam/ai-interview-coach

1. Project at a Glance

AI Interview Coach is a full-stack web application intended to support interview preparation through CV-aware practice, answer evaluation, and actionable feedback. It combines a browser-based interface, a Python API backend, relational data storage, and a planned AI/NLP integration layer.

Category	Technology / Details
Project type	Full-stack AI-powered web application
Frontend	React + Vite
Backend	Python + FastAPI
ASGI server	Uvicorn
Database	PostgreSQL
ORM / migrations	SQLAlchemy / Alembic
API style	HTTP REST-style endpoints; JSON and multipart uploads where applicable
Version control	Git + GitHub
Deployment attempt	Hostless (backend and PostgreSQL resource)

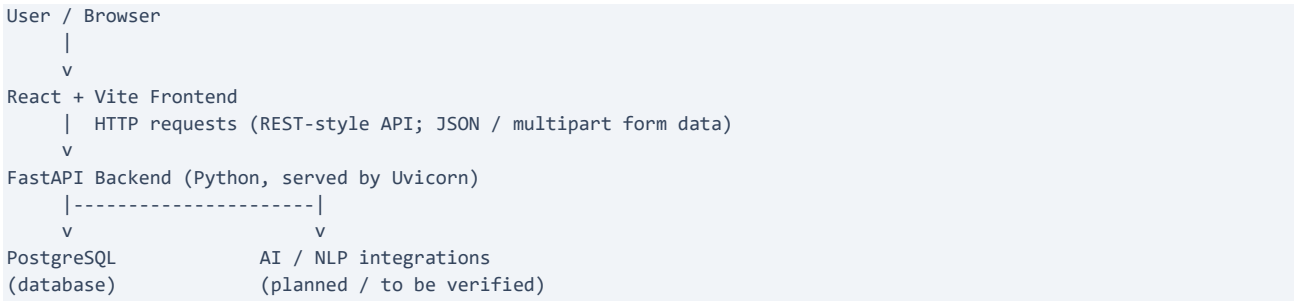
2. Problem and Proposed Solution

Many students, recent graduates, and job seekers have limited access to realistic interview practice and personalized feedback. AI Interview Coach aims to provide structured practice that can use a candidate's CV and target role to tailor questions and help improve answers.

Core user journey

1. Upload a CV and select a target job role.
2. Start an interview practice session and respond to questions.
3. Submit answers for evaluation.
4. Review scores, strengths, weaknesses, and improvement suggestions.
5. Revisit interview results and history.

3. High-Level Architecture



The frontend presents the interface and sends requests to the backend. FastAPI handles request validation and application logic and communicates with PostgreSQL through the database layer. AI services/models are intended to support personalized question generation and answer assessment; verify each integration's implementation status in the current code.

4. Languages, Frameworks, and Libraries

Layer	Technology	Role
Language	Python	Backend logic, API endpoints, configuration, database integration
Language	JavaScript / JSX	Interactive frontend components and client-side behavior
Web UI	React	Component-based user interface
Build tool	Vite	Frontend development server and production build

Layer	Technology	Role
Backend framework	FastAPI	HTTP API and request/response handling
ASGI server	Uvicorn	Runs the FastAPI application
Validation/config	Pydantic	Data validation and settings management
Database ORM	SQLAlchemy	Python database access and ORM
Migrations	Alembic	Versioned database schema changes
Styling	CSS	Responsive layout, themes, and visual styling
Navigation / animation	React Router / Framer Motion	Listed in project overview; confirm exact dependencies in package.json

5. Database and Persistence

PostgreSQL is the database selected for the current hosting setup. SQLAlchemy provides the Python database-access layer, and Alembic manages schema migrations. The application obtains its database URL through environment variables.

- Expected data areas include user accounts, interview sessions, submitted answers, and evaluation results.
- The hosted database is separate from local development databases.
- Apply migrations only to the intended hosted database after checking DATABASE_URL.

DEPLOYMENT STATUS

A PostgreSQL resource has been created on Hostless. The backend deployment was attempted, but the public URL returned “Invalid host header” during testing. The hosted application still requires configuration and end-to-end verification.

6. APIs and AI / NLP Components

Application API

The frontend and backend communicate through HTTP API requests. REST-style endpoints return structured responses, commonly JSON. CV upload endpoints may use multipart/form-data. Confirm exact route names and schemas from API_SPECIFICATION.md and FastAPI OpenAPI documentation once the service is running.

- Authentication and account management.
- CV upload and processing.
- Interview creation and question retrieval.
- Answer submission.
- Evaluation, feedback, results, and interview history.

AI / NLP technologies in the capstone proposal

Technology / Service	Proposed purpose	Status note
OpenAI API and/or Amazon Bedrock	Generate role- and CV-aware questions and ideal-answer suggestions	Proposed; verify provider integration in code and environment
Whisper	Speech-to-text transcription	Proposed; verify implementation
Sentence Transformers	Semantic similarity scoring of answers	Proposed; verify implementation
BERT / DistilBERT	NLP-based answer assessment	Proposed; verify implementation

These services are identified in the capstone proposal and should not be described as active production integrations unless they are configured and invoked by the current application.

7. Repository Structure

The repository separates frontend, backend, project documentation, and supporting configuration.

```
ai-interview-coach/
├── frontend/
│   ├── public/
│   └── src/
```

```
│       ├── animations/
│       ├── components/
│       ├── context/
│       ├── data/
│       ├── hooks/
│       ├── pages/
│       ├── services/
│       ├── styles/
│       ├── utils/
│       ├── App.jsx
│       └── main.jsx
│
├── backend/
│   ├── app/
│   │   ├── core/
│   │   │   └── config.py
│   │   ├── services/
│   │   │   └── interview_service.py
│   │   └── main.py
│   ├── alembic/
│   ├── tests/
│   ├── storage/
│   ├── requirements.txt
│   └── .env.example
│
├── API_SPECIFICATION.md
├── README.md
└── AI_Capstone_Project_Proposal_Team147
```

This is a high-level structure based on available project context, not an exhaustive filesystem export. Confirm exact paths and filenames against the latest GitHub commit.

Directory responsibilities

Path	Purpose
frontend/src/components/	Reusable UI components
frontend/src/pages/	Application screens and routed pages
frontend/src/context/	Shared application state
frontend/src/hooks/	Reusable React hooks
frontend/src/services/	Frontend API and service calls
frontend/src/data/	Static data and question-bank content
frontend/src/styles/	CSS and theme styling
backend/app/main.py	FastAPI application entry point
backend/app/core/	Application configuration and core settings
backend/app/services/	Interview and business logic
backend/alembic/	Database migration configuration and revisions
backend/tests/	Automated tests
backend/storage/	Local file storage, where configured
backend/requirements.txt	Python dependencies

8. Testing and Quality Assurance

Previously reported test results from the development environment:

Check	Reported result	Notes
Backend test suite	260 passed, 8 skipped	Skipped tests included PostgreSQL migration tests when TEST_DATABASE_URL was not configured
Frontend production build	Successful	Reported for the earlier code version

These are previously reported results, not a new test run performed while preparing this handbook. Rerun tests and the frontend build against the final commit before submission.

9. Version Control and Deployment

- Git is used for version control.
- GitHub hosts the source repository and main branch.
- Repository: https://github.com/Mennatullah-essam/ai-interview-coach
- Hostless is being used to test deployment of the backend and PostgreSQL resource.
- The frontend still needs its own deployment and a configured backend base URL.
- Production configuration requires correct DATABASE_URL, JWT_SECRET, ALLOWED_HOSTS, and CORS_ORIGINS values.

SECURITY REMINDER

Never commit real credentials to GitHub or share database connection strings and JWT secrets in documentation. Store them as deployment environment variables.

10. Instructor Summary

AI Interview Coach is a full-stack interview-preparation platform built around a React + Vite frontend, a Python + FastAPI backend, and PostgreSQL persistence. Uvicorn runs the API server, SQLAlchemy provides database access, and Alembic manages schema migrations. The capstone proposal includes AI/NLP options such as Whisper, Sentence Transformers, BERT/DistilBERT, and OpenAI API or Amazon Bedrock for speech transcription, semantic answer evaluation, and personalized question generation. Deployment is still being verified, and the active status of each external AI integration should be confirmed against the code before the final demonstration.

Reference

GitHub repository: https://github.com/Mennatullah-essam/ai-interview-coach

Capstone proposal: AI_Capstone_Project_Proposal_Team147

Confirm exact file paths, dependency versions, API routes, and active AI-provider configuration against the latest repository before formal assessment.